2 POINT SOFT

GAMEPAD VIBRATION SYSTEM

Complete user manual and practical reference



Engine	Unity Input System
API	C#, components, profiles, and UnityEvents
Manual version	1.0 — English

Standard gamepad rumble through Unity Input System. Includes One Shot, Pulsed, Continuous, per-device mixing, local targeting, reusable profiles, and selective stopping.

Contents

1. Introduction 4

2. Requirements and compatibility 4

3. Installation 4

Quick test . . . 5

4. Core concepts 5

Low Frequency . . . 5

High Frequency . . . 5

Duration . . . 5

Pulses and Interval . . . 5

Fade Out . . . 5

Global Intensity . . . 5

5. Playback modes 5

Important Continuous rule . . . 6

6. No-code workflow: GamepadVibration component 6

Main fields . . . 6

Public methods . . . 6

UnityEvent example . . . 6

7. GamepadVibrationTrigger 7

No-code Continuous Hazard . . . 7

8. Built-in presets 7

9. GamepadVibrationSettings 8

10. GamepadVibrationManager 8

Historical convenience API . . . 8

Play settings or presets . . . 8

Stop All . . . 8

Mixing . . . 8

11. GamepadVibrationHandle and ownership 9

Complete Continuous example . . . 9

12. Custom profiles 9

Profile Manager . . . 9
Use a profile from C# . . . 10
Continuous profile . . . 10

13. Optional generated typed API **10**

14. Targeting and local multiplayer **11**

Current Gamepad . . . 11
Explicit Gamepad . . . 11
PlayerInput . . . 11
Recommendation . . . 11

15. Practical examples **11**

Gunshot recoil . . . 11
Modified explosion preset . . . 11
Continuous damage hazard . . . 12
UI through components . . . 12

16. Demo scenes **12**

GamepadVibrationDemo . . . 12
GamepadVibrationLocalMultiplayerDemo . . . 12
GamepadVibrationProfilesDemo . . . 12

17. Disconnections and safe stopping **12**

18. Troubleshooting **13**

No vibration . . . 13
Wrong local player vibrates . . . 13
Continuous vibration does not stop . . . 13
Generated member is missing . . . 13
DualSense . . . 13
Vibration continues after changing state . . . 13

19. Quick reference: important types **14**

20. Product limits **14**

21. Integration checklist **14**

Visual product workflows **15**

Publisher: 2 Point Soft **Main namespace:** `TwoPointSoft.GamepadVibration` **Technology:** Unity Input System **Document:** User manual and practical reference

> Gamepad Vibration System provides standard low- and high-frequency motor rumble. It does not include adaptive triggers or platform-specific advanced haptics.

1. Introduction

Gamepad Vibration System is a standalone solution for adding controller vibration to Unity games through `UnityEngine.InputSystem`. It can be used without code, from UnityEvents, with reusable components, through `ScriptableObject` profiles, or directly from C#.

Main features:

- **One Shot**, **Pulsed**, and **Continuous** playback.
- Independent low- and high-frequency motor control.
- Protected built-in presets and reusable custom profiles.
- Direct C# API and UnityEvent-compatible methods.
- Per-device mixing of simultaneous requests.
- Handles that stop only the effects they own.
- Targeting through `Gamepad` or `PlayerInput`.
- Disconnection cleanup.
- Unscaled-time playback.
- Profile Manager and optional typed API.
- Three demonstration scenes.

The Manager is created automatically when the first vibration is requested. You do not need to add it to a scene manually.

2. Requirements and compatibility

Item	Requirement or scope
Unity	Validated with Unity 6000.3.15f1
Input	Unity Input System 1.19.0
Device	A controller recognized by `UnityEngine.InputSystem.Gamepad`
Demo UI	Unity UGUI
Render pipeline	Runtime is independent of Built-in, URP, and HDRP
DualSense	Standard rumble validated over USB on Windows

The runtime does not depend on Game Creator, TextMesh Pro, or third-party packages. Bluetooth behavior and individual device compatibility depend on Unity Input System and the target platform.

3. Installation

1. Import `Assets/2PointSoft/GamepadVibrationSystem` into the project.
2. Open **Window > Package Manager**.

3. Confirm that **Input System** is installed.
4. If Unity requests it, enable the new Input System and restart the Editor.
5. Connect a supported gamepad.
6. Open `Demo/Scenes/GamepadVibrationDemo.unity` and enter Play Mode.

The demo UI also requires Unity UGUI.

Quick test

Add the component from:

```
Add Component > 2 Point Soft > Gamepad Vibration
```

Configure an effect, connect `Play()` to a button or event, and enter Play Mode.

4. Core concepts

Low Frequency

Controls the heavy motor. It is useful for explosions, machinery, heavy impacts, engines, and events that should feel weighty.

High Frequency

Controls the faster motor. It is useful for gunfire, light impacts, UI feedback, alerts, and quick textures.

Both intensities use the `0` to `1` range.

Duration

The duration, in seconds, of each pulse. Playback uses unscaled time, so it continues to update when `Time.timeScale` changes.

Pulses and Interval

`Pulses` controls the number of repetitions. `Interval` controls the pause between them.

Fade Out

Gradually reduces both motor intensities during each pulse.

Global Intensity

A final multiplier for both motors. It preserves their relative balance while changing the overall strength.

5. Playback modes

Mode	Behavior	Typical use
One Shot	Runs one finite pulse	Impact, jump, UI selection
Pulsed	Repeats pulses with intervals	Alert, heartbeat, burst
Continuous	Runs indefinitely until Stop is called	Engine, hazard, machinery

Important Continuous rule

Continuous playback has no finishing duration. Stop it with the returned `GamepadVibrationHandle`, with `Stop()` on its owning component, or with a device-targeted Stop operation.

6. No-code workflow: GamepadVibration component

`GamepadVibration` is the main designer-facing component.

Main fields

Field	Description
Source	Uses built-in/custom values or a `GamepadVibrationProfile`
Preset	Chooses a protected preset or Custom values
Playback Mode	One Shot, Pulsed, or Continuous
Target Mode	`CurrentGamepad` or `PlayerInput`
Player Input	Optional PlayerInput for local multiplayer
Low / High Frequency	Motor intensities
Duration	Duration of each pulse
Pulses	Number of pulses
Interval	Pause between pulses
Fade Out	Gradual intensity reduction
Global Intensity	Final multiplier

Public methods

```
public void Play();
public void Stop();
public void ApplyPreset(GamepadVibrationPreset newPreset);
```

`Stop()` stops only requests created by that component. When disabled or destroyed, the component cleans up its own effects without stopping unrelated vibration.

UnityEvent example

1. Add `GamepadVibration` to a GameObject.
2. Configure a preset or custom values.
3. In a `Button.onClick`, drag the GameObject into the event.
4. Select `GamepadVibration > Play()`.
5. To stop Continuous playback, connect another event to `GamepadVibration > Stop()`.

This also works with Animation Events, custom gameplay events, and serialized UnityEvents.

7. GamepadVibrationTrigger

`GamepadVibrationTrigger` plays and stops effects through 3D trigger colliders.

Fields:

- **Vibration:** the controlled `GamepadVibration` component.
- **Trigger Event:** play on `OnEnter` or `OnExit`.
- **Required Tag:** limits which objects can activate it; empty accepts any tag.
- **Trigger Once:** allows one activation.
- **Stop On Opposite Event:** stops on the opposite event; recommended for Continuous.

No-code Continuous Hazard

1. Add a collider and enable **Is Trigger**.
2. Add `GamepadVibration` and choose **Continuous**.
3. Add `GamepadVibrationTrigger`.
4. Set **Trigger Event = OnEnter**.
5. Enable **Stop On Opposite Event**.
6. Use the `Player` tag on the entering object.

The effect starts on entry and stops on exit.

To allow another activation when Trigger Once is enabled:

```
trigger.ResetTrigger();
```

8. Built-in presets

The package includes:

- `LightHit`
- `HeavyHit`
- `Explosion`
- `Heartbeat`
- `Alert`
- `Hazard`
- `UISelect`
- `UIConfirm`

Direct example:

```
using TwoPointSoft.GamepadVibration;

GamepadVibrationPresets.HeavyHit.Play();
```

Create a variant without modifying the preset:

```
GamepadVibrationSettings custom = GamepadVibrationPresets.HeavyHit.With(
    duration: 0.35f,
    globalIntensity: 0.75f,
    fadeOut: true);

GamepadVibrationHandle handle = custom.Play();
```

Built-ins are protected values. To edit a persistent variation, copy one into a custom profile from the Profile Manager.

9. GamepadVibrationSettings

`GamepadVibrationSettings` is an immutable structure containing validated values.

```
GamepadVibrationSettings settings = new GamepadVibrationSettings(
    lowFrequency: 0.65f,
    highFrequency: 0.90f,
    duration: 0.18f,
    pulses: 3,
    interval: 0.08f,
    fadeOut: true,
    globalIntensity: 0.85f);

GamepadVibrationHandle handle = settings.Play();
```

Intensities are clamped to `0–1`; duration and interval cannot be negative; pulses cannot be lower than one.

Main methods:

```
settings.Play();
settings.Play(gamepad);
settings.Play(playerInput);
settings.With(...);
```

10. GamepadVibrationManager

`GamepadVibrationManager` is the static core of the system. It tracks requests, mixes effects, updates motors, and cleans up disconnected devices.

Historical convenience API

```
GamepadVibrationManager.Play(
    low: 0.5f,
    high: 0.8f,
    duration: 0.2f,
    fadeOut: true);
```

Calls without an explicit target use `Gamepad.current`.

Play settings or presets

```
GamepadVibrationHandle hit =
    GamepadVibrationManager.Play(GamepadVibrationPresets.HeavyHit);

GamepadVibrationHandle alert =
    GamepadVibrationManager.PlayPreset(GamepadVibrationPreset.Alert);
```

Stop All

```
GamepadVibrationManager.StopAll();
GamepadVibrationManager.StopAll(gamepad);
GamepadVibrationManager.StopAll(playerInput);
```

Use `StopAll()` as a global or emergency action. During normal gameplay with overlapping effects, prefer stopping the owning handle or component.

Mixing

Simultaneous requests are added per device and clamped to the valid range before being sent to the controller. One effect does not automatically replace another. Each gamepad maintains an independent mix.

11. GamepadVibrationHandle and ownership

A `GamepadVibrationHandle` represents a sequence started from code.

```
GamepadVibrationHandle handle = GamepadVibrationPresets.Alert.Play();

// Later:
handle.Stop();
```

Stopping a handle does not cancel unrelated requests. This is essential when UI, combat, vehicles, and environmental systems can vibrate simultaneously.

Complete Continuous example

```
using UnityEngine;
using UnityEngine.InputSystem;
using TwoPointSoft.GamepadVibration;

public sealed class EngineRumbleExample : MonoBehaviour
{
    private GamepadVibrationHandle engineHandle;

    public void StartEngine()
    {
        Gamepad target = Gamepad.current;
        if (target == null) return;

        engineHandle?.Stop();
        engineHandle = GamepadVibrationManager.PlayContinuous(
            low: 0.45f,
            high: 0.15f,
            target: target);
    }

    public void StopEngine()
    {
        engineHandle?.Stop();
        engineHandle = null;
    }

    private void OnDisable()
    {
        StopEngine();
    }
}
```

12. Custom profiles

`GamepadVibrationProfile` is a reusable `ScriptableObject`. It centralizes a vibration feel for use across multiple scenes and objects.

Create one from the asset menu:

```
Create > 2 Point Soft > Gamepad Vibration Profile
```

You can also use:

```
Tools > 2 Point Soft > Vibration Profile Manager
```

Profile Manager

The Profile Manager supports:

- searching and filtering profiles;
- inspecting protected built-ins;
- creating from scratch;

- copying a built-in;
- copying an existing profile;
- editing through a working copy;
- explicit Save;
- resetting unsaved changes;
- Play Mode preview;
- duplicating and deleting custom profiles;
- generating the optional C# API.

Save Profile saves the working copy. **Reset Unsaved Changes** restores the state from the most recent explicit Save.

Use a profile from C#

```
using UnityEngine;
using TwoPointSoft.GamepadVibration;

public sealed class SwordExample : MonoBehaviour
{
    [SerializeField] private GamepadVibrationProfile swordAttack;

    public void Attack()
    {
        GamepadVibrationHandle handle = swordAttack?.Play();
    }
}
```

Continuous profile

```
private GamepadVibrationHandle idleHandle;

public void StartIdle()
{
    idleHandle?.Stop();
    idleHandle = engineIdleProfile.Play();
}

public void StopIdle()
{
    idleHandle?.Stop();
    idleHandle = null;
}
```

13. Optional generated typed API

Generation creates:

```
Assets/GamepadVibrationGenerated/GamepadVibrationCustom.cs
```

It is optional. Profiles, components, and demos work without this file.

Example:

```
using TwoPointSoft.GamepadVibration;
using TwoPointSoft.GamepadVibration.Generated;

GamepadVibrationCustom.SwordAttack.Play();
GamepadVibrationCustom.ShotgunRecoil.Play(playerInput);

GamepadVibrationHandle idle =
    GamepadVibrationCustom.EngineIdle.Play(gamepad);
idle.Stop();
```

After creating, renaming, or deleting profiles, click **Generate API** or **Update API** and wait for Unity to compile.

14. Targeting and local multiplayer

Current Gamepad

APIs without a target use `Gamepad.current`. This is suitable for single-player projects and quick tools.

```
GamepadVibrationPresets.LightHit.Play();
```

Explicit Gamepad

```
Gamepad target = Gamepad.current;
if (target != null)
{
    GamepadVibrationPresets.Explosion.Play(target);
}
```

PlayerInput

```
using UnityEngine.InputSystem;

[SerializeField] private PlayerInput playerInput;

public void DamagePlayer()
{
    GamepadVibrationPresets.HeavyHit.Play(playerInput);
}
```

The system resolves the first `Gamepad` paired with that `PlayerInput`. An invalid explicit target does not fall back to another player's controller.

Recommendation

Do not use `Gamepad.all[playerIndex]` as permanent player identity. Its order may change when devices connect or disconnect. Store the gamepad that joined or use each player's `PlayerInput`.

15. Practical examples

Gunshot recoil

```
public void Fire()
{
    GamepadVibrationSettings recoil = new GamepadVibrationSettings(
        lowFrequency: 0.35f,
        highFrequency: 0.95f,
        duration: 0.08f,
        pulses: 1,
        fadeOut: true);

    recoil.Play();
}
```

Modified explosion preset

```
public void Explode(Gamepad target)
{
    GamepadVibrationSettings explosion =
        GamepadVibrationPresets.Explosion.With(globalIntensity: 0.65f);

    explosion.Play(target);
}
```

Continuous damage hazard

```
private GamepadVibrationHandle hazardHandle;

public void EnterHazard(Gamepad target)
{
    hazardHandle?.Stop();
    hazardHandle = GamepadVibrationManager.PlayContinuous(0.55f, 0.35f, target);
}

public void ExitHazard()
{
    hazardHandle?.Stop();
    hazardHandle = null;
}
```

UI through components

Configure `UISelect` on one component and connect `EventTrigger.PointerEnter` or your navigation system to `Play()`. Use a separate `UIConfirm` component for confirmation.

16. Demo scenes

GamepadVibrationDemo

```
Demo/Scenes/GamepadVibrationDemo.unity
```

Shows components, UnityEvents, presets, direct API calls, triggers, shooting, impacts, pulsed effects, and Continuous machinery.

GamepadVibrationLocalMultiplayerDemo

```
Demo/Scenes/GamepadVibrationLocalMultiplayerDemo.unity
```

Supports up to four connected controllers. Each slot stores the exact gamepad that joined. Disconnected slots are released so replacement controllers can join.

GamepadVibrationProfilesDemo

```
Demo/Scenes/GamepadVibrationProfilesDemo.unity
```

Included examples:

1. **Sword Attack** — One Shot.
2. **Shotgun Recoil** — Pulsed.
3. **Engine Idle** — Continuous with separate Start and Stop.

17. Disconnections and safe stopping

The Manager removes requests associated with disconnected devices. Invalid explicit targets do not redirect vibration to another controller.

When the final request ends, a short stop-confirmation window is used before releasing the device. This improves standard HID rumble reliability without keeping permanent ownership.

Best practices:

- Store and stop Continuous handles.
- Stop owned effects in `OnDisable`.
- Use targeted Stop in multiplayer.

- Avoid `StopAll()` for canceling one effect.
- Do not keep `Gamepad.all` indices as player identity.

18. Troubleshooting

No vibration

- Confirm that Input System is installed and active.
- Connect the controller and press a button to establish `Gamepad.current`.
- Confirm that the device supports standard rumble through Unity Input System.
- Check that Low, High, and Global Intensity are greater than zero.
- Enter Play Mode before using Preview.

Wrong local player vibrates

Use `Play(playerInput)`, an explicit `Gamepad`, or the `PlayerInput` target mode. Do not identify players using changing device indices.

Continuous vibration does not stop

Keep the handle and call `handle.Stop()`, or call `Stop()` on the component that started the effect.

Generated member is missing

Open the Profile Manager, click **Update API** or **Generate API**, and wait for Unity to finish compiling.

DualSense

Standard rumble was validated over USB on Windows. Bluetooth depends on platform and Unity Input System support. Adaptive triggers and advanced haptics are outside the product's scope.

Vibration continues after changing state

Ensure that the owner stops its handle in `OnDisable`, `OnDestroy`, when leaving the gameplay state, or when exiting a trigger. For Continuous triggers, enable **Stop On Opposite Event**.

19. Quick reference: important types

Type	Responsibility
`GamepadVibrationManager`	Core, mixing, targeting, Stop, and disconnects
`GamepadVibration`	Reusable UnityEvent-compatible component
`GamepadVibrationTrigger`	Trigger-based play and stop
`GamepadVibrationSettings`	Immutable validated settings
`GamepadVibrationPresets`	Built-in presets
`GamepadVibrationProfile`	Reusable custom profile asset
`GamepadVibrationHandle`	Ownership and selective stopping
`GamepadVibrationGeneratedProfile`	Runtime type used by generated API
`GamepadVibrationSource`	Values/preset or profile source
`GamepadVibrationPlaybackMode`	OneShot, Pulsed, or Continuous
`GamepadVibrationTargetMode`	CurrentGamepad or PlayerInput

20. Product limits

Gamepad Vibration System provides standard two-motor rumble through Unity Input System. It does not include:

- adaptive triggers;
- advanced PS5-specific haptics;
- networking;
- network player pairing;
- guaranteed support for devices Unity does not recognize as `Gamepad`;
- a mandatory URP or HDRP dependency.

Final compatibility depends on the controller, connection type, operating system, input backend, and Unity Input System version.

21. Integration checklist

- Input System is installed and active.
- The controller is recognized by Unity.
- The correct playback mode is selected.
- Continuous effects keep their handle or owning component.
- Multiplayer uses `PlayerInput` or stable `Gamepad` references.
- UnityEvents are connected to `Play()` and `Stop()`.
- Profiles are explicitly saved.
- The generated API is updated only if the project uses it.
- Demos have been tested with target hardware.

Visual product workflows

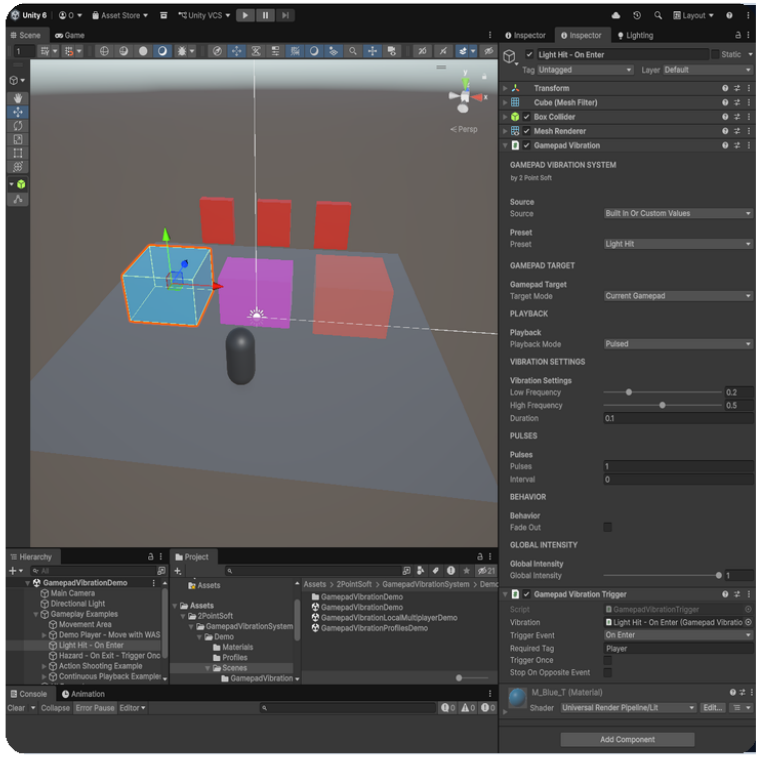
These sheets summarize the system's primary workflows. They use landscape pages to preserve interface and diagram readability.

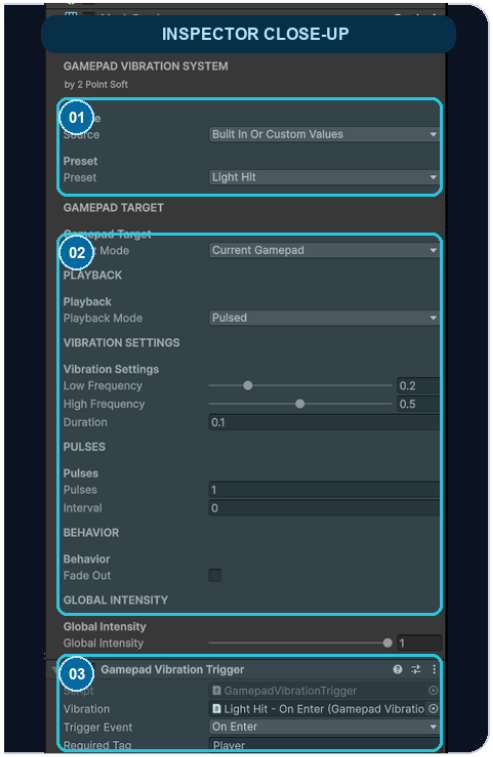
INSPECTOR WORKFLOW

SET UP VIBRATION WITHOUT CODE

Choose an effect, adjust its response, and trigger it directly from a GameObject.

GAMEPAD VIBRATION SYSTEM





NO-CODE SETUP

01 Choose the source
Use a built-in preset, custom values, or a reusable vibration profile.

02 Shape the response
Select the target and playback mode, then tune both motors and timing.

03 Trigger from gameplay
Play on Enter or Exit, filter by tag, and optionally stop on the opposite event.

DESIGNER-FRIENDLY
Configure effects in the Inspector and connect them to reusable gameplay triggers.

COMPONENT WORKFLOW PRESETS, CUSTOM VALUES, AND PROFILES

UNITY INPUT SYSTEM

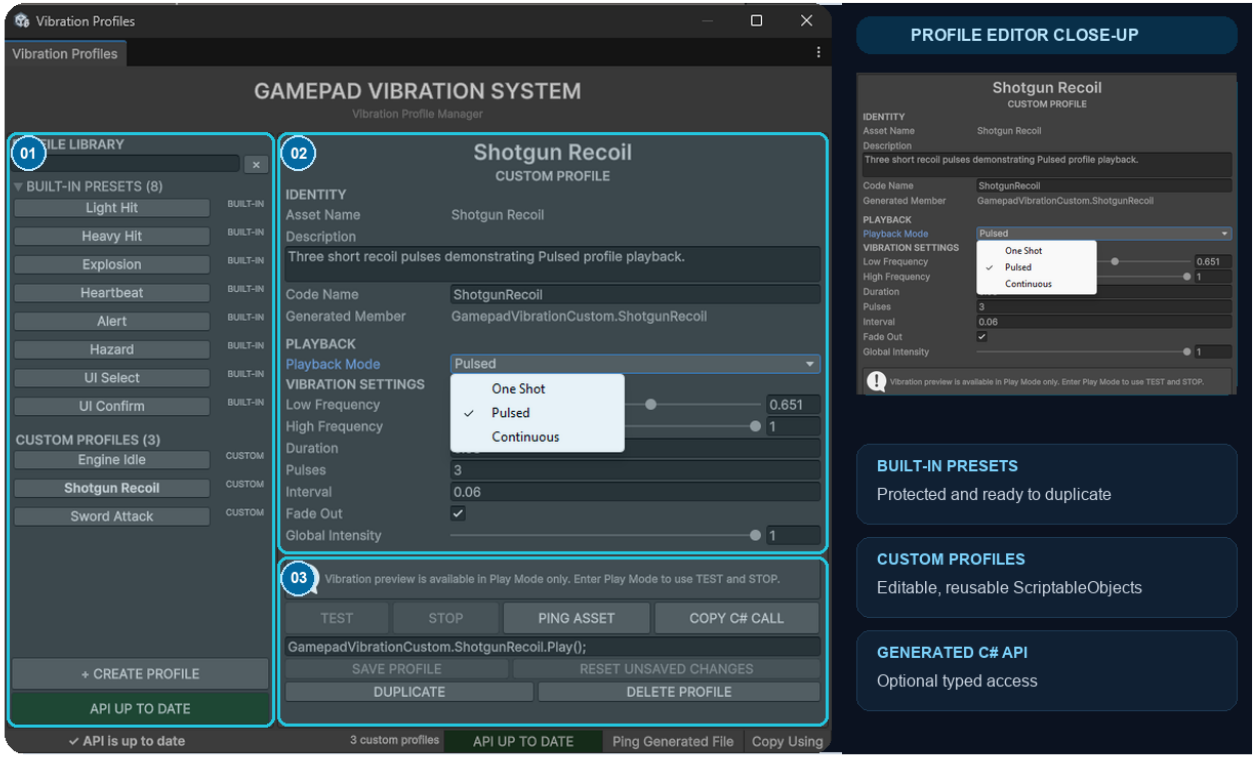
Inspector workflow: choose the source, shape the response, and connect it to gameplay events.

PROFILE MANAGER

CREATE AND MANAGE VIBRATION PROFILES

Build reusable effects, preview them, and access them from components or C#.

GAMEPAD VIBRATION SYSTEM



PROFILE EDITOR CLOSE-UP

BUILT-IN PRESETS
Protected and ready to duplicate

CUSTOM PROFILES
Editable, reusable ScriptableObjects

GENERATED C# API
Optional typed access

PROFILE WORKFLOW

01 **Organize profiles**
Browse protected built-in presets and searchable custom profiles in one library.

02 **Design the effect**
Choose One Shot, Pulsed, or Continuous, then tune motors, timing, and Fade Out.

03 **Save and use**
Preview in Play Mode, duplicate profiles, ping the asset, or copy a ready C# call.

OPTIONAL GENERATED API
Profiles work directly as assets.
Code generation is never required.

CENTRAL PROFILE LIBRARY • BUILT-IN PRESETS AND CUSTOM PROFILES

UNITY INPUT SYSTEM

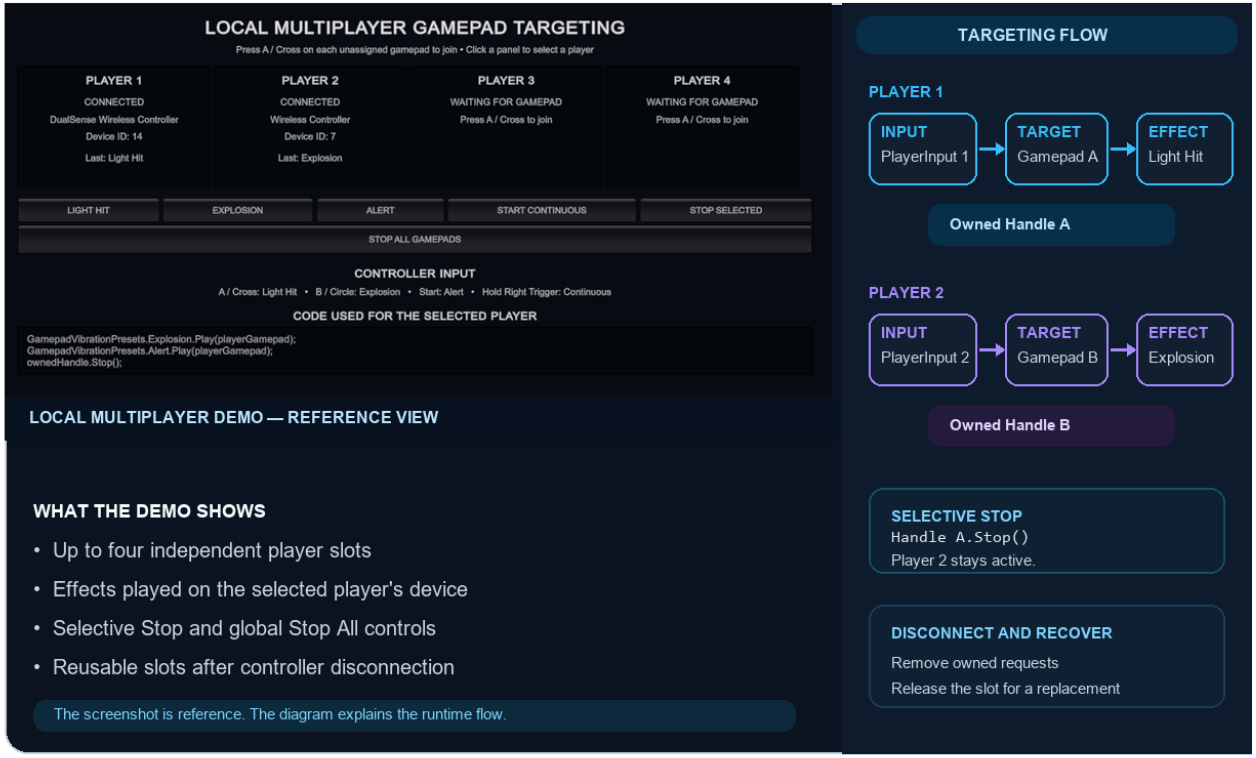
Profile Manager: organize presets and custom profiles, edit the effect, and access usage actions.

LOCAL MULTIPLAYER

TARGET VIBRATION PER PLAYER

Each joined player keeps an independent gamepad target and vibration handle.

GAMEPAD VIBRATION SYSTEM



MULTIPLAYER FLOW

01 **Join and bind**
Each player joins with a specific gamepad. The project keeps that device reference.

02 **Target the player**
Pass the player's Gamepad or PlayerInput so rumble reaches only that device.

03 **Stop independently**
Each playback returns an owned handle. Stopping it does not affect other players.

NO INDEX-BASED TARGETING
Joined device references remain stable when other controllers connect or leave.

PER-PLAYER TARGETING • GAMEPAD AND PLAYERINPUT SUPPORT

UNITY INPUT SYSTEM

Local multiplayer: each player keeps an independent gamepad, effect, and stopping handle.